

DESIGN REPORT

End-to-End Analytics Engineering Data Pipeline

Eric Ondeniyi · Data Engineer

1. Overview

This report documents the design of an end-to-end analytics engineering pipeline that ingests data from a public REST API, lands it in a PostgreSQL OLTP layer, replicates changes in near real time into ClickHouse via Debezium change data capture (CDC), and transforms raw events into clean, analytics-ready and ML-ready datasets using a layered dbt model (staging → mart). The pipeline is orchestrated end-to-end, containerized for single-command startup, deployed through a GitHub Actions CI/CD pipeline with automated testing, and monitored through a Prometheus/Grafana observability stack.

Worked example used throughout this report: a public products/orders REST API (e.g. DummyJSON) is treated as an upstream e-commerce system. Products and orders are pulled into PostgreSQL, and downstream consumers (analysts, ML feature pipelines) query ClickHouse for near-real-time sales and inventory analytics. The same architecture generalises to any public REST API with a comparable entity shape (a few related resources, incremental updates over time).

2. Architecture Diagram

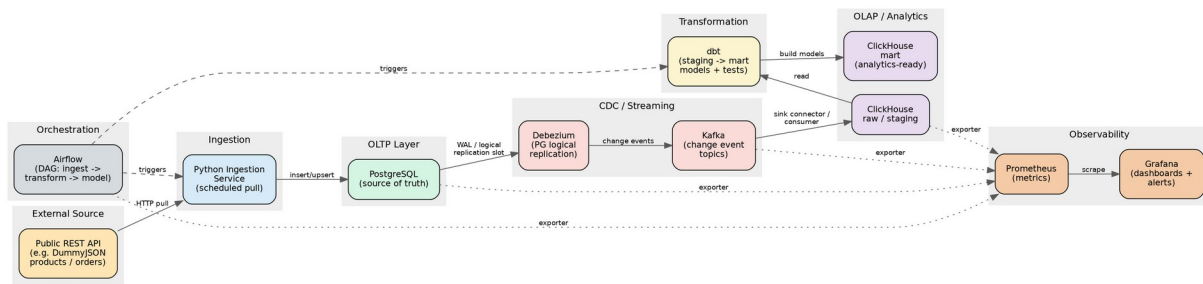


Figure 1. End-to-end architecture: ingestion → OLTP → CDC → OLAP → transformation, with orchestration and observability spanning every layer.

2.1 Component Rationale

Component	Why this choice
Ingestion service	A small Python service polls the public REST API on a schedule and upserts records into PostgreSQL. Kept outside Airflow's own executor so it can be retried/scaled independently; Airflow only triggers and monitors it.
PostgreSQL (OLTP)	Acts as the system of record and the CDC source. Chosen over ingesting straight into ClickHouse because it gives transactional upserts, constraints, and a stable WAL for Debezium to read — ClickHouse is not designed for row-level upserts at ingestion time.
Debezium	Reads PostgreSQL's write-ahead log via logical replication and emits row-level change events (insert/update/delete) with before/after state — this is what makes replication “near real time” rather than batch-pollled.
Kafka	Durable, replayable buffer between Debezium and ClickHouse. Decouples produce and consume rates, and allows the ClickHouse sink (or a consumer) to reprocess a topic from any offset if a downstream schema changes.
ClickHouse (raw/staging)	Change events land largely as-is (append-only, including the CDC operation type and timestamp) so the raw layer is a faithful, replayable copy of upstream

	changes — nothing is thrown away before staging.
dbt	Owns all business logic: staging models standardize types and flag deletes; mart models denormalize, aggregate, and apply ClickHouse-specific physical design. dbt tests enforce not-null/uniqueness/referential checks at each layer boundary.
ClickHouse (mart)	Analytics-ready tables and materialized views, modeled specifically for ClickHouse's MergeTree family rather than a generic star schema — see §3 for engine/partition/order choices.
Airflow	Orchestrates the full path (ingest → wait for CDC lag to settle → dbt run → dbt test) as a single DAG with clear task-level retries and SLAs.
Prometheus / Grafana	Exporters on Postgres, Kafka, ClickHouse and Airflow feed Prometheus; Grafana dashboards and alert rules surface pipeline health, CDC lag, and data freshness in one place (see §4).

3. Data Model / Schema

The schema follows a strict staging → mart layering. Raw CDC events are never queried directly by consumers; every analytics-facing table is built by dbt.

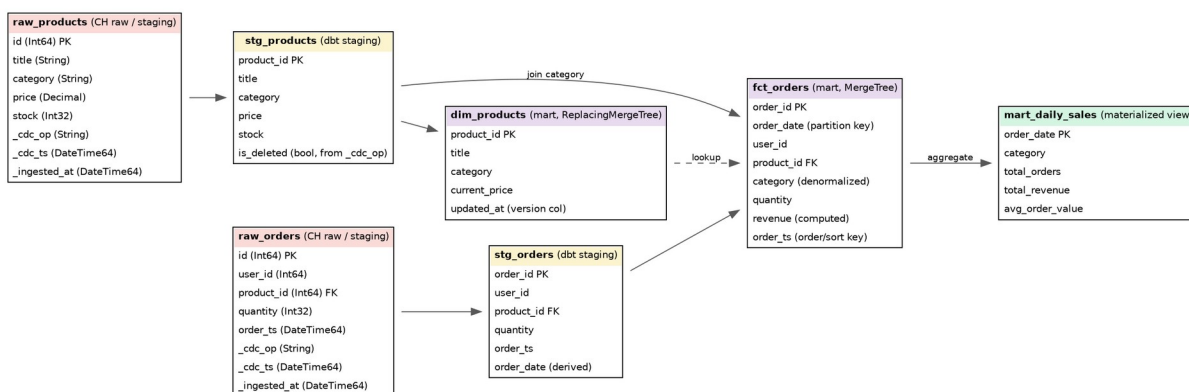


Figure 2. Schema flow from raw CDC tables through dbt staging models into ClickHouse mart tables and a pre-aggregated materialized view.

3.1 Layer Definitions

- **raw_*** (ClickHouse, MergeTree, append-only): one row per CDC event, including `_cdc_op` (insert/update/delete) and `_cdc_ts`. Nothing is overwritten — this layer is the audit trail and replay source.
- **stg_*** (dbt staging models, views or lightweight tables): type casting, column renaming, and collapsing CDC history down to current-row semantics (`is_deleted` flags, latest-value resolution).
- **dim_ / fct_** (dbt mart models, ClickHouse tables): denormalized, business-facing tables optimized for the query patterns analysts and ML feature pipelines actually run (filter by date, group by category).
- **mart_daily_sales** (materialized view): pre-aggregated rollup so dashboards don't scan `fct_orders` on every load.

3.2 ClickHouse-Specific Design Choices

Decision	Rationale
Table engine — fct_orders	MergeTree. Standard choice for large, append-heavy fact data with no need for the last-write-wins semantics ReplacingMergeTree provides.
Table engine — dim_products	ReplacingMergeTree(updated_at). Products get updated in place upstream; this engine keeps only the latest version per <code>product_id</code> on merge, matching

	dimension-table semantics without manual dedup logic.
Partition key — fct_orders	toYYYYMM(order_date). Keeps partitions to a manageable size for a table that grows daily, and lets ClickHouse prune whole months for date-ranged queries.
Order/sort key — fct_orders	(order_date, category, order_id). Matches the dominant query shape (date-ranged, category-filtered aggregation) so ClickHouse can skip granules instead of scanning the full partition.
Materialized view — mart_daily_sales	Built with AggregatingMergeTree-style incremental aggregation so dashboard queries hit a small pre-aggregated table instead of re-scanning fct_orders on every request.
Raw layer engine	MergeTree ordered by (_cdc_ts), append-only. Optimized for high-throughput sequential writes from the Kafka consumer, not for point lookups.

4. Observability Design

Observability spans both platform health (are the systems up and keeping pace) and data health (is the data itself trustworthy and fresh). Both are treated as first-class, not an afterthought bolted on at the end.

4.1 What Is Monitored

Signal	What's tracked
Pipeline health	Airflow task success/failure rate, task duration trend, DAG run SLA misses (via the Airflow StatsD/Prometheus exporter).
CDC lag	Debezium connector lag metrics (source offset vs. last committed offset) and Kafka consumer group lag on the CDC topics — the leading indicator that “near real time” is actually being met.
Data freshness	Max(_ingested_at) / max(_cdc_ts) per raw table, exposed as a custom metric and alerted on if it exceeds an expected staleness threshold (e.g. no new events in 15 minutes during business hours).
Data quality	dbt test pass/fail counts per run (not-null, uniqueness, relationships, accepted-values), surfaced as a Prometheus metric via a small dbt-artifacts exporter so failing tests show up on the same dashboard as infra metrics, not buried in CI logs.
Resource usage	Postgres connection count and replication slot lag, Kafka broker disk/throughput, ClickHouse query latency and merge/part counts, container CPU/memory via cAdvisor.

4.2 Tooling Choice

Prometheus + Grafana were chosen over alternatives (e.g. a managed SaaS APM) because every component in this stack (Postgres, Kafka, Debezium, ClickHouse, Airflow) already has a mature, well-supported Prometheus exporter, and Grafana lets pipeline-health and data-quality metrics live on the same dashboard rather than split across a logging tool and a separate data-quality tool. Alerting rules in Prometheus (Alertmanager) cover the failure modes that need paging — CDC lag breach, DAG SLA miss, repeated dbt test failure — while Grafana dashboards stay the default view for day-to-day health checks.

5. Scaling & Extension

Growth vector	How the design extends
---------------	------------------------

Ingestion throughput	Move from a single scheduled poller to a fan-out of paginated/parallel API workers, or switch the source system to push webhooks where the API supports them, removing polling latency entirely.
CDC / Kafka throughput	Increase Kafka topic partitions and consumer parallelism; scale Debezium via Kafka Connect's distributed mode instead of a single connector instance.
ClickHouse write volume	Move from single-node to a ClickHouse cluster with sharding (by a hash of order_id or user_id) and replication; introduce a buffer table or Kafka engine table in front of MergeTree to smooth write bursts.
Query volume / concurrency	Add more materialized views for the next tier of dashboard queries as they emerge; consider a read-replica shard group so heavy ad-hoc analyst queries don't compete with the ingestion path.
Orchestration complexity	As more sources are added, split into per-source DAGs with clear dependency sensors rather than one monolithic DAG, and move shared logic into Airflow task groups or custom operators.
Schema evolution	Introduce a schema registry (e.g. for Kafka/Debezium's Avro or JSON schema support) so upstream schema changes fail fast and visibly instead of silently breaking dbt models downstream.